

Fly.io — o que é, o que custa e o que precisa saber antes de decidir

Levantado em 19/08/2026 direto da documentação oficial de preços, sizing, CPU, volumes e gestão de custo. A decisão está no [ADR-015](#); este doc é a explicação de apoio.

 **Revisado em 19/08 após parecer externo (GPT).** A primeira versão continha um erro grave — afirmava que dava para configurar limite de gasto e alerta, o que **não existe no Fly** — e usava preço da região errada. Sete correções aplicadas; as principais estão marcadas com  **Corrigido** ao longo do texto.

Em uma frase

Micro-VMs cobradas por segundo, em **São Paulo**, rodando a nossa imagem Docker, por **~\$7,50/mês**, sem precisar cuidar de máquina Linux.

 **Corrigido:** a primeira versão dizia "*sem ninguém precisar administrar servidor*", o que vende demais. O que sai da nossa conta é o **servidor** — SO, patches, firewall, Coolify. **Continuam nossos:** deploy, configuração da Machine, volume, segredos, métricas e resposta a incidente. É bem menos trabalho que uma VPS, não é zero.

1. O modelo de cobrança: não existe plano

É **pay-as-you-go puro**. Nas palavras deles: "*Plans get complicated, so we just charge based on usage.*" Paga-se por segundo de máquina ligada, GB de disco provisionado e GB de tráfego de saída.

O que a nossa configuração custa — preço de gru

 **Corrigido:** a versão anterior usava o preço da região base. **Machine tem preço por região**, e São Paulo é mais cara.

Item	Preço em gru	Nosso caso
shared-cpu-1x com 1 GB	\$0,00000279/s (~\$7,23/mês)	1 máquina 24/7
Volume	\$0,15/GB/mês	1 GB = \$0,15
Egress América do Sul	\$0,04/GB	tráfego de painel, centavos
Certificado TLS	primeiros 10 grátis	\$0
IPv4 dedicado	\$2/mês	só se necessário

Total: ~\$7,50/mês.

O quanto São Paulo custa a mais, para referência:

shared-cpu-1x	IAD (mais barata)	GRU	Diferença
512 MB	\$0,00000123/s	\$0,00000156/s	+27%
1 GB	\$0,00000220/s	\$0,00000279/s	+27%
2 GB	\$0,00000413/s	\$0,00000524/s	+27%

Ou seja: a latência de São Paulo custa ~27% a mais de compute. Em dinheiro, ~\$1,50/mês.

Comparando com o que foi avaliado

Opção	Custo	Gerenciado?	São Paulo?
Fly.io	~\$7,50	✓	✓
VPS (Hostinger KVM 2)	~\$14 na renovação	✗	✓
Render Standard	\$25	✓	✗

✓ **Corrigido:** a frase anterior dizia que o Fly era "o único gerenciado e o único em São Paulo". Errado — o Render também é gerenciado, só não tem região no Brasil. O correto é: **o Fly é o único que combina ambiente gerenciado com São Paulo.**

Ainda assim, **preço não é o critério** — é latência. Se a medição não mostrar ganho, a escolha cai mesmo custando um terço.

● Consumo não tem teto — e o Fly NÃO oferece proteção

✓ **Corrigido** — este era o erro mais grave da versão anterior. Ela mandava "configurar limite de gasto e alerta". Isso não existe no Fly. A documentação oficial de gestão de custo diz, textualmente:

"We don't support billing alerts (yet), so budget accordingly."

E sobre as franquias gratuitas:

"Free allowances don't cap your bill... If you go over, we'll bill you."

Não há teto rígido, não há alerta, não há corte automático. O que dá para fazer é o que eles próprios recomendam:

1. **Conferir a fatura parcial no painel** ("current month to date bill") com regularidade
— é retrospectivo, não preventivo
1. **Orçar pelo pior caso**, ou seja, pelo custo de tudo ligado 24/7 — não contar com economia de hibernação (que a gente desligou de propósito, ver seção 6)
1. **Manter uma única Machine e não ligar autoscaling** — o controle real é rodar menos máquinas, ou menores
1. `billing@fly.io` em caso de cobrança inesperada

Isso é uma diferença concreta em relação a plano fixo, e precisa entrar na decisão com o nome certo: no Render você não pode gastar mais que o plano; no Fly, pode.

Outras notas de preço

- Não existe mais free tier, só um trial — conferir os detalhes ao criar a conta.
- Suporte pago começa em \$29/mês, mais caro que a infraestrutura inteira. O suporte da comunidade atende neste porte.

2. O que ele oferece de máquina

Tipo	vCPU	RAM possível
shared-cpu-1x a 8x	1 a 8	256 MB × N até 2 GB × N
performance-1x a 8x	1 a 8	2 GB × N até 8 GB × N

Nossa configuração é `shared-cpu-1x` com 1 GB. O teto desse tipo é 2 GB, então há para onde crescer sem trocar de categoria.

512 MB está descartado desde o começo: foi o que matou o container no Render Free durante o cron do TikTok, em 15/08.

3. ⚠ A pegadinha do "shared": 6,25% de um núcleo

Shared e performance rodam no **mesmo hardware, com o mesmo clock**. A diferença é quanto tempo de execução se recebe a cada período de 80 ms:

```
shared      ->  5 ms / 80 ms = 6,25% de um núcleo (sustentado)
performance -> 80 ms / 80 ms = 100%
```

Mas ele acumula crédito. O tempo ocioso vira saldo de burst, até 500 segundos de CPU cheia guardados. Estourou o saldo, a máquina é **estrangulada nos 6,25%** até recarregar. Em máquinas maiores as cotas somam: `shared-cpu-2x` recebe 10 ms por período.

Por que isso importa para o NEXO

✅ **Corrigido** — erro de análise da versão anterior. Ela dizia que os syncs "queimam saldo" porque duram 20–60 s. Duração não é consumo de CPU. Os nossos syncs passam a maior parte do tempo *esperando* resposta da Amazon, do Mercado Livre, do TikTok e do Supabase — isso é espera de rede, que praticamente não gasta processador.

O que realmente consome CPU é o **parsing e a materialização** dos dados recebidos, que é uma fração pequena desse tempo. Ou seja: o cenário é bem mais favorável ao `shared-cpu-1x` do que a versão anterior deste doc sugeria.

A referência que tranquiliza: o Render Free entrega **0,1 CPU sustentado e zero burst**, e o app sobrevive nele hoje. O Fly dá um pouco menos de base e 500 s de rajada que o Render nunca deu.

🔥 **Mas segue sendo medição, não premissa.** Só o painel de throttling responde. Se o saldo acabar com frequência, o caminho é `performance-1x` — e aí o custo sobe muito, a vantagem de preço evapora e a **VPS volta a fazer sentido**.

4. ⚠ A pegadinha do volume: disco local, sem réplica

Documentação do Fly: um volume é *"uma fatia de um NVMe no mesmo servidor físico da Machine"*. Eles comparam com o disco interno de um notebook.

Compartilhar entre máquinas	✗ um volume, uma máquina
Replicação	✗ nenhuma — se o NVMe falhar, a aplicação cai
Snapshot	diário automático, 5 dias (configurável 1–60)
Encolher	✗ só dá para aumentar
Tamanho máximo	500 GB

A própria documentação avisa que o snapshot **não deve ser o backup principal**.

✅ **Corrigido:** a versão anterior citava a recomendação deles de "*provisionar pelo menos dois volumes*" como se fosse a solução. **Não é** — volumes **não se replicam entre si**. Dois volumes só ajudam se houver **duas Machines** e **alguma estratégia de replicação de dados** (LiteFS, ou o dado morar num Postgres gerenciado). Provisionar um segundo volume sozinho não compra disponibilidade nenhuma.

O ponto fraco de verdade

`DATA_DIR` guardando contas OAuth e custos em disco local é a maior fragilidade desta fase — mais séria que CPU ou preço.

Para um piloto é aceitável, **com backup**. Mas **esses dados precisam migrar para o Postgres antes de o NEXO depender definitivamente do Fly**. Isso vale igualmente no Render e em VPS — não é defeito do Fly, é dívida nossa que o Fly torna mais visível.

5. O que muda no dia a dia

Não existe painel para configurar infraestrutura. É CLI (`fly`) e o arquivo `fly.toml` , versionado no repo. Para nós é melhor — configuração revisável junto do código — mas é diferente do Render, onde se clica.

6. As duas configurações que não podem ser esquecidas

Ambas já estão no `fly.toml` do repo, com comentário explicando.

```
auto_stop_machines = false
```

O Fly hiberna máquina ociosa por padrão, e é daí que vem boa parte da economia da plataforma. **A gente desligou de propósito:** o cache do NEXO vive na memória do processo ([ADR-002](#)) e a conciliação de tarifas roda em background. Máquina que hiberna perde o cache e mata trabalho em andamento — seria reintroduzir, por configuração, o mesmo problema que fez a Vercel ser rejeitada.

Consequência assumida: **pagamos o mês cheio**. Os ~\$7,50 já refletem isso — e é também o que os próprios docs mandam fazer ao orçar (pelo custo "always-on").

```
[mounts] com DATA_DIR
```

O app grava custos e contas OAuth em disco. Sem volume montado, **cada deploy apaga esses dados**.

7. Como subir

Detalhes de build e das pegadinhas do Docker em [docker.md](#).

```
# 1. instalar o CLI e autenticar
curl -L https://fly.io/install.sh | sh
fly auth login

# 2. criar o volume ANTES do primeiro deploy
fly volumes create nexo_data --region gru --size 1

# 3. deploy
wsl bash scripts/fly-deploy.sh

# 4. segredos de runtime (lista completa em render.yaml)
fly secrets set DATABASE_URL=... LWA_CLIENT_SECRET=...
```

⚠ **Nunca rodar `fly deploy cru`.** Ele não passa os `--build-arg`, e a imagem sobe com as credenciais do Supabase vazias — o app funciona, o health check passa, e só a tela de login denuncia. O script existe para isso.

8. O piloto — e o que ele precisa provar

A decisão aprovada é testar, não migrar. O cutover só acontece depois que a medição sustentar a escolha.

Antes de subir

- [] Destruir o limite do Supabase (559 MB de 500 MB, subindo ~7 MB/dia)
- [] Registrar domínio próprio e cadastrar nas allowlists de Shopee e TikTok — sem

isso o OAuth dos dois canais quebra na troca de host, **em qualquer cenário**

- [] Conferir **capacidade em gru** e o que o trial inclui
- [] Aceitar conscientemente que **não há teto de gasto**, e definir uma rotina de

conferir a fatura parcial no painel

O piloto

Uma máquina `shared-cpu-1x` de 1 GB, **autostop desligado, sem autoscaling**, com o Render servindo produção o tempo todo, rodando todos os syncs por pelo menos uma semana.

O que medir antes de trocar o DNS

Medida	Por que decide
Latência contra o Render	É a justificativa inteira. Não melhorou, decisão cai
Saldo de burst e throttling	Se acabar sempre, <code>shared-cpu-1x</code> não serve
Memória	512 MB já matou o app uma vez
Persistência do volume entre deploys	<code>DATA_DIR</code> não pode sumir
Custo parcial no painel	Única proteção que existe

Se a máquina compartilhada aguentar, o Fly resolve por ~\$7,50. Se o burst acabar sempre e for preciso `performance-1x`, a vantagem de preço encolhe muito e a VPS volta a fazer mais sentido — por isso a decisão de cutover fica para depois da medição, não agora.

8.5. Como desenvolver sem rodar nada local

Contexto: a máquina de desenvolvimento aqui tem `C:` cheio e RAM apertada. Três medidas tiram o trabalho pesado dela.

1. Build nos servidores do Fly (padrão)

`fly deploy` usa o Docker local se encontrar um. No primeiro deploy foi o que aconteceu — o build rodou nesta máquina. Para não acontecer:

```
ws1 bash scripts/fly-deploy.sh          # --remote-only por padrão
ws1 bash scripts/fly-deploy.sh --local  # força build local, se quiser
```

Com build remoto, o Fly compila nos servidores dele e a máquina local só envia os arquivos. Zero RAM e zero disco seus.

2. Um app de staging que dorme

Um segundo app (`nexo-staging`) com `auto_stop_machines = true` — ligado, ao contrário da produção.
🔥 Aqui a hibernação é desejável. O motivo de desligá-la em produção é o cache em memória ([ADR-002](#)); staging não precisa de cache quente. A máquina dorme quando ninguém usa e acorda no primeiro acesso.
Custo: usando ~2h por dia, fica em torno de \$0,60/mês em vez de \$7.
Fluxo que isso destrava:

```
edita o código → deploy no staging → testa no navegador → aprova → deploy em nexo
```

Nada roda na máquina local.
⚠ Staging não deve apontar para o banco de produção. Enquanto não houver um banco separado, tratar o staging como somente-leitura e nunca disparar sync manual por ele.

3. Testes automatizados no GitHub Actions

`npm test` roda na nuvem a cada push, de graça — a conta já usa Actions para o cron ([ADR-003](#)).

Deploy: Fly é manual, Render é automático

	Render	Fly
Gatilho	<code>git push</code> na main (<code>autoDeploy: true</code>)	comando, quando você quiser
Controle	nenhum — push errado vai ao ar	você escolhe a hora
Estratégia	—	<i>rolling</i> : a máquina nova só entra depois de passar no health check

Dá para automatizar com GitHub Actions depois. Não foi feito agora de propósito: com o Render ainda em produção, dois hosts subindo sozinhos sobre o mesmo banco é confusão.

9. Resultado do primeiro deploy (19/08/2026, ~18h30)

App `nexo` no ar em `gru` : `https://nexo.fly.dev` · imagem de 62 MB · volume de 1 GB criptografado com snapshot diário · health check `1 total`, `1 passing` .

Latência medida — a justificativa do ADR-015, confirmada

Rota	Fly (<code>gru</code>)	Render (Oregon)
<code>/api/health</code> (sem banco)	0,074 – 0,080 s	0,210 – 0,525 s
<code>/login</code> (toca o Supabase)	0,094 – 0,147 s	0,208 – 0,514 s

2 a 5× mais rápido — e muito mais estável. O Fly variou ~6 ms entre rodadas; o Render variou de 210 a 525 ms. A instabilidade do Render é provavelmente o que faz o painel parecer travado de vez em quando.

Validado de ponta a ponta (19/08, ~20h)

A vendedora fez login em `nexo.fly.dev` e o **dashboard carregou com os dados reais**. Isso fecha a cadeia inteira: imagem Docker → máquina em `gru` → Supabase Auth → Postgres → renderização.

Antes disso, o que dava para provar de fora era parcial e vale registrar por quê:

- `/api/health` não toca o banco — só devolve `{ok:true}` . Health check passando não prova conectividade com Postgres.
- Todas as rotas de dados (`/api/orders` , `/api/sales` , `/api/products`) respondem `**401` antes** de consultar o banco.

🔗 **Lição:** um health check que não toca a dependência crítica dá falsa segurança. Vale considerar um `/api/health?deep=1` que faça um `SELECT 1` — hoje o app responde "saúdável" mesmo se o banco estiver inacessível.

Também confirmado nos logs: o volume montou **correto** em `/data` (1 GB, uid 100/gid 101), que era um dos riscos listados.

O que ficou provado e o que não

- ✅ Capacidade em `gru` existe (era dúvida do checklist) · ✅ o `Dockerfile` do repo sobe no Fly sem ajuste · ✅ 29 segredos aplicados por `fly secrets set` · ✅ o gate de "verificação de conta" **não bloqueia** o deploy depois do cartão cadastrado.
- ❌ **Ainda não medido:** saldo de burst e throttling sob carga de sync, memória sob carga, persistência do volume entre deploys, custo real acumulado. São os itens que exigem a semana de piloto.
- ⚠️ `nexo.fly.dev` aponta para o banco de produção. Render e Fly estão vivos ao mesmo tempo, sobre os mesmos dados. Não é problema — o Render segue sendo produção — mas evita disparar sync manual pelos dois ao mesmo tempo.
- ⚠️ **OAuth não funciona em `nexo.fly.dev`** : o domínio não está nas allowlists de Shopee e TikTok. Leitura funciona; reautorizar canal, não. É o pré-requisito do domínio próprio.

Nota de método

Este doc afirmou três coisas erradas sobre o Fly antes de serem verificadas: que dava para configurar limite de gasto (não dá), que duração de sync equivale a consumo de CPU (não equivale) e que o trial permitia testar sem cartão (a tela exige verificação para deployar). Duas foram corrigidas por revisão externa, uma pela própria tela. **Preço, limite e gate de plataforma são para ler na fonte antes de escrever, não depois.**

Relacionado: [ADR-015](#) · [docker.md](#) · [infra-decisao-hospedagem.md](#)